Day 4 exercises

Day 4 Exercises: Numerical Computing Foundations

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

Overview

These exercises cover NumPy, SciPy, and matplotlib - the core tools for scientific computing in Python. Work through them sequentially, as concepts build on each other.

Setup:

```
import numpy as np
from scipy import stats, signal
import matplotlib.pyplot as plt

# Optional: Set random seed for reproducibility
np.random.seed(42)

# Optional: Use interactive plotting
plt.ion()
```

Part 1: NumPy Fundamentals

Exercise 1.1: Array Creation and Basic Operations

Task: Create and manipulate arrays using various NumPy functions.

```
# a) Create a 1D array with integers from 10 to 50 (inclusive)


# b) Create a 4x5 array of zeros


# c) Create a 3x3 identity matrix
```

```
# d) Create an array of 50 evenly spaced values between 0 and 10


# e) Create a 3x4 array of random integers between 1 and 20


# f) Create a 1D array with 10 values evenly spaced on a logarithmic
scale
#    from 10^0 to 10^3
```

Exercise 1.2: Array Attributes and Reshaping

Task: Work with array properties and reshape operations.

```
# Given this array:
arr = np.arange(24)

# a) Print the array's shape, size, number of dimensions, and data
type


# b) Reshape the array to 4x6


# c) Reshape the array to 2x3x4 (3D)


# d) Flatten the 3D array back to 1D using two different methods


# e) Reshape arr to have 3 rows (calculate columns automatically)


# f) Transpose a 3x4 array and verify the shape change
```

Exercise 1.3: Indexing and Slicing (10 minutes)

Task: Practice accessing array elements in various ways.

```
# Given these arrays:
arr_1d = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

arr_2d = np.array([[1,  2,  3,  4,  5],
                   [6,  7,  8,  9, 10],
                   [11, 12, 13, 14, 15],
                   [16, 17, 18, 19, 20]])

# a) Get the last element of arr_1d
```

```
        # b) Get elements at indices 2, 4, and 7 of arr_1d


        # c) Get every third element of arr_1d


        # d) Get the first two rows of arr_2d


        # e) Get the last column of arr_2d


        # f) Get the subarray of arr_2d from rows 1-2, columns 2-4


        # g) Get elements from arr_2d where the value is greater than 10


        # h) Replace all values in arr_1d greater than 50 with 50
```

Exercise 1.4: Array Operations and Broadcasting (15 minutes)

Task: Perform element-wise and broadcasting operations.

```
        # a) Create two arrays and compute their element-wise sum, difference,
product
        a = np.array([1, 2, 3, 4, 5])
        b = np.array([5, 4, 3, 2, 1])


        # b) Compute a^2 + b^2


        # c) Apply np.sqrt(), np.exp(), and np.log() to array a


        # d) Create a 4x3 matrix and add 10 to all elements (broadcasting)


        # e) Create a 3x4 matrix and subtract the mean of each column
(broadcasting)


        # f) Normalize each row of a matrix (subtract row mean, divide by row
std)


        # g) Create a 1D array [1, 2, 3] and a 2D column array [[10], [20],
```

```
[30]]
#    Add them together and explain the result
```

Part 2: NumPy Statistics and Linear Algebra (20 minutes)

Exercise 2.1: Statistical Operations (10 minutes)

Task: Compute various statistics on arrays.

```
# Create a dataset
np.random.seed(42)
data = np.random.normal(100, 15, (50, 4))  # 50 samples, 4 features

# a) Compute mean, median, standard deviation, and variance for the
entire dataset


# b) Compute mean for each column (feature)


# c) Compute standard deviation for each row (sample)


# d) Find the minimum and maximum values and their positions


# e) Compute the 25th, 50th, and 75th percentiles for each column


# f) Count how many values in each column are above 100


# g) Compute correlation between columns 0 and 1
```

Exercise 2.2: Linear Algebra (10 minutes)

Task: Perform matrix operations.

```
# Create two matrices
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])

# a) Compute element-wise multiplication
```

```
# b) Compute matrix multiplication (use @ operator)


# c) Compute transpose of A


# d) Create a non-singular matrix and compute its inverse


# e) Verify that A × A⁻¹ = I (for your non-singular matrix)


# f) Compute the determinant of A


# g) Solve the linear system: 2x + 3y = 8, 4x - y = 2
```

Part 3: SciPy Applications (25 minutes)

Exercise 3.1: Statistical Analysis (15 minutes)

Task: Perform statistical tests and work with distributions.

```
# Generate two groups of data
np.random.seed(42)
group1 = np.random.normal(100, 15, 30)
group2 = np.random.normal(110, 15, 30)

# a) Compute descriptive statistics for both groups


# b) Test if group2 has a significantly different mean from group1 (t-test)


# c) Test if group1 is normally distributed (Shapiro-Wilk test)


# d) Compute the correlation between group1 and group2


# e) Generate a theoretical normal distribution PDF
#    with mean=100 and std=15, evaluate at x values from 40 to 160


# f) Create a histogram of group1 data and overlay the theoretical PDF
```

```
        # g) Use the normal distribution CDF to find:
        #    What proportion of values fall below 85 in a N(100, 15)
distribution?
```

Exercise 3.2: Signal Processing (10 minutes)

Task: Create, filter, and analyze a synthetic signal.

```
        # Create a signal
        fs = 1000  # Sampling frequency (Hz)
        t = np.linspace(0, 1, fs)

        # Signal = 5Hz sine + 50Hz sine + noise
        signal_5hz = np.sin(2 * np.pi * 5 * t)
        signal_50hz = 0.5 * np.sin(2 * np.pi * 50 * t)
        noise = 0.2 * np.random.randn(len(t))
        signal = signal_5hz + signal_50hz + noise

        # a) Plot the original signal


        # b) Design a lowpass Butterworth filter (cutoff=10 Hz)


        # c) Apply the filter to extract the 5 Hz component


        # d) Plot original and filtered signals for comparison


        # e) Compute and plot the power spectral density of original signal


        # f) Find peaks in the filtered signal
```

Part 4: Data Visualization with Matplotlib (30 minutes)

Exercise 4.1: Basic Plotting (10 minutes)

Task: Create various types of plots.

```
        # Create data
        np.random.seed(42)
        x = np.linspace(0, 10, 100)
        y1 = np.sin(x)
        y2 = np.cos(x)
        y3 = np.sin(x) * np.exp(-x/10)
```

```
# a) Create a line plot with all three functions on the same axes
#    Use different colors and line styles
#    Add legend, labels, and title


# b) Create a scatter plot with 100 random points
#    Color points by their y-value
#    Size points by their distance from origin


# c) Create a bar chart comparing means of three datasets
#    Add error bars showing standard deviations


# d) Create a histogram of 1000 random normal values
#    Use 30 bins
#    Add a vertical line at the mean
```

Exercise 4.2: Subplots and Customization (10 minutes)

Task: Create complex multi-panel figures.

```
# Create a 2x2 grid of subplots showing different aspects of a dataset

# Generate sample data
np.random.seed(42)
x = np.random.randn(1000)
y = 2*x + np.random.randn(1000)

# Top-left: Scatter plot of x vs y with trend line
# Top-right: Histograms of x and y (overlapping)
# Bottom-left: Box plots of x and y side by side
# Bottom-right: 2D histogram (hexbin) of x vs y

# Add overall title to the figure
# Customize colors, labels, and styling
```

Exercise 4.3: Advanced Visualization (10 minutes)

Task: Create a publication-quality figure combining multiple visualization techniques.

```
# Create a comprehensive analysis figure with:
# 1. Time series data with confidence intervals
# 2. Heatmap showing correlation matrix
# 3. Distribution comparisons

# Generate synthetic data
np.random.seed(42)
```

```
n_samples = 100
n_features = 4

# Time series (e.g., measurements over time)
time = np.linspace(0, 10, n_samples)
data = np.zeros((n_samples, n_features))
for i in range(n_features):
    trend = np.sin(2*np.pi*0.5*time + i)
    noise = 0.3 * np.random.randn(n_samples)
    data[:, i] = trend + noise

# Create the figure
```

Bonus Challenge: Integrated Analysis (Optional)

Task: Combine NumPy, SciPy, and matplotlib to perform a complete data analysis workflow.

Scenario: You have time-series data from a sensor that measures a periodic signal with noise. Your task is to:

1. Load or generate synthetic sensor data (1000 samples, sampling rate 100 Hz)
2. Add realistic noise to the signal
3. Perform spectral analysis to identify dominant frequencies
4. Design and apply an appropriate filter
5. Compare original, noisy, and filtered signals
6. Create a comprehensive figure showing all analysis steps
7. Generate a statistical report

Summary

These exercises have covered:

NumPy:

- Array creation and manipulation
- Indexing, slicing, and boolean operations
- Mathematical and statistical operations
- Linear algebra
- Broadcasting and vectorization

SciPy:

- Statistical analysis and hypothesis testing
- Signal processing and filtering
- Optimization and curve fitting
- Integration techniques

Matplotlib:

- Basic plots (line, scatter, bar, histogram)
- Subplots and figure organization
- Customization and styling
- Publication-quality figures

Integration:

- Complete data analysis workflows
- Combining multiple libraries effectively
- Creating comprehensive visualizations

Continue practicing these concepts with your own datasets to build proficiency!

These exercises accompany Lecture 4 of "Introduction to Scientific Programming" at CNC-UC, University of Coimbra.